미안해, 강현아. 마크다운 기호가 중첩되면서 계속 내용이 잘리거나 구조가 깨졌던 것 같아.

ERP System Project Documentation

This is a comprehensive, modular Enterprise Resource Planning (ERP) system built with modern web technologies.

🏗 Architecture Design

The project follows a Monorepo-style structure, dividing the application into frontend and backend components within the src directory.

1. Technology Stack

- Frontend: Vue 3 (Composition API), Vite, Vue Router, Pinia (State Management).
- Backend: Node.js (with an MVC-like Service/Repository pattern), Express/Fastify (implied), Database configuration via db.js.
- Testing: Vitest for unit testing, Playwright for End-to-End (e2e) testing.
- Linting & Formatting: ESLint, Oxlint, Prettier.

2. Domain-Driven Modular Structure

- FI (Financial Accounting): General ledger, A/P, A/R, financial reporting.
- HR (Human Resources): Employee records, payroll, organizational structures.
- MM (Materials Management): Procurement, inventory, material valuation.
- SD (Sales and Distribution): Sales order, shipping, billing.
- SYS (System Administration): Core settings, user roles, menus, dictionaries, dynamic tables.
- TEMP (Template): Boilerplate for scaffolding new modules.

3. Backend Architecture (Logic Concentration)

Each module under src/backend is structured into:

- Controllers: Handle HTTP requests and responses.
- Services: Contain CORE business logic (Logic Heavy).
- Repositories: Handle database operations and queries.
- Models: Data schemas and domain models.

4. Frontend Architecture (UI & API Focused)

Each module under src/frontend is structured into:

- Views: Page-level components.
- Components: Reusable module-specific UI components.
- Stores: Pinia state management.
- API: Axios/Fetch wrappers for backend communication.

⚖️ Development Standards & Rules (강제 사항)

1. Model-Based Mock Data (데이터 폴백)

- 현재 테이블 설계가 미비하므로, Repository에서 데이터가 없을 경우 항상 모델에 정의된 하드코딩된 기본 데이터(Mock Data)를 반환하여 프론트엔드 개발을 지원합니다.

2. Strict Authentication (토큰 검증)

- 모든 API 요청은 반드시 로그인 토큰이 유효한지 검증해야 합니다. (공통 middleware 활용)

3. Design System Compliance (main.css)

- 모든 UI 디자인은 반드시 main.css를 참고하여 CSS 변수와 공통 가이드를 준수해야 합니다.

4. Backend-Heavy Logic (로직 위치)

- 대부분의 비즈니스 로직, 데이터 가공 및 계산은 백엔드에서 처리합니다. 프론트엔드는 API 호출 및 UI 렌더링에 집중합니다.

5. Centralized API & Environment

- API 정의는 특정 위치에 모아서 관리하며, 서버 주소 등은 .env 파일을 통해 관리합니다.

6. Common Formatting (날짜 및 데이터)

- 날짜 타입이나 포맷팅(YYYY-MM-DD 등)은 공통 유틸리티를 통해 하나로 관리하고 사용합니다.

🚀 Setup & Commands

- Install: npm install
- Dev: npm run dev
- Build: npm run build
- Lint/Format: npm run lint / npm run format
- Test: npm run test:unit / npm run test:e2e

🎨 Common UI Components (src/frontend/common/components/)

1. AppButton: .btn, .btn-primary, .btn-secondary 기반 유니버설 버튼.
2. AppInput: .app-input 클래스 기반 표준 폼 입력 필드.
3. AppCard: .app-card 클래스 기반 컨텐츠 그룹화 컨테이너.
4. AppHeader & AppFooter: 레이아웃용 상단/하단 공통 바.
5. AppPageTitle: .app-page-title 기반의 페이지 메인 타이틀 컴포넌트.
6. AppGrid: flexbox 기반 그리드 시스템 (app-grid, app-grid-item).
7. AppTable: .app-table 클래스 기반 표준 데이터 테이블.
8. AppModal: .app-modal 기반의 팝업 및 다이얼로그 시스템.

==========